JavaScript Notes

Hoisting & Scope · Switch Statements · while / do-while Loops

1. Hoisting & Variable Scope

Hoisting is JavaScript's behavior of moving **declarations** to the top of their scope before code runs. With `var`, the declaration is hoisted but not the value - so the variable exists from the start of the function/script, but holds `undefined` until the line that assigns it actually executes.

Global vs. function scope

A `var` declared inside a function is local to that function - it does not overwrite a global variable of the same name. Once the function ends, that local copy disappears and the outer/global variable is unaffected.

```
var a = 10;

function test() {
  var a = 20;
  console.log(a);   // 20  (local var)
}
test();
console.log(a);      // 10  (global var, untouched)
```

Hoisting inside a function

Even though the `var a` declaration appears after the `console.log` inside the function, JavaScript hoists the declaration to the top of the function. So the log happens *after* the declaration but *before* the assignment - the result is `undefined`, not an error and not the outer value.

```
a = 10;

function test() {
  console.log(a);   // undefined (hoisted local 'a', not yet assigned)
  var a = 20;
}
test();
console.log(a);      // 10
```

Key takeaway: `var` is function-scoped (not block-scoped), and its declaration is hoisted to the top of that function - only the declaration, not the assigned value. This is why relying on hoisting is confusing; in modern code, prefer `let`/`const`, which are block-scoped and throw an error if used before declaration instead of silently being `undefined`.

2. Switch Statement

A `switch` compares one value against several possible `case` values and runs the matching block. `break` stops execution from "falling through" into the next case. `default` runs when nothing matches.

```
var days = ['sun', 'mon', 'tues', 'wed', 'thurs', 'fri', 'sat'];
var day = days[new Date().getDay()];
```

```
switch (day) {
  case 'sat':
  case 'sun':
    console.log('holiday: ' + day);
    break;
  case 'mon':
    console.log('working day: ' + day);
    break;
  case 'fri':
    console.log('half day: ' + day);
    break;
  default:
    console.log('invalid');
}
```

Note: `new Date().getDay()` returns a number (0 = Sunday ... 6 = Saturday), which is used here as the index into the `days` array to get a matching string like 'mon' or 'sat'. Also watch variable naming - reusing the same name for the array index and the mapped day string (e.g. both called `day`) can shadow the earlier value; give them distinct names such as `dayIndex` and `dayName` for clarity.

3. while and do...while Loops

while loop

A `while` loop checks its condition *before* each pass. If the condition is false the very first time, the loop body never runs.

```
var i = 1;

while (i <= 10) {
  console.log(i, 'test');
  i++;
}
```

Careful: writing `while (i >= 10)` starting from `i = 1` means the condition is false immediately, so the loop body never executes at all. Also make sure the loop variable is updated (e.g. `i++`) inside the body - forgetting it, or writing a condition that's never true, causes the loop to be skipped entirely (and forgetting it with a condition that starts true would cause an infinite loop instead).

do...while loop

A `do...while` loop checks its condition *after* each pass, so the body always runs at least once - even if the condition is false from the start.

```
var i = 0;

do {
  console.log(i, 'test');
  i++;
} while (i < 10);
```

Careful: `while (i > 10)` starting from `i = 0` will still run the body once (that's the defining feature of do-while), print '0 test', then stop because 0 > 10 is false. If the goal was to print 0 through 9, the condition should be `i < 10` instead.

while vs. do...while - the core difference

	while	do...while
Condition checked	Before the loop body	After the loop body
Runs at least once?	No - can run zero times	Yes - always runs once minimum
Typical use	When you're not sure the body should run at all	When the body must run at least once (e.g. menus, input pro

Quick recap

- var declarations are hoisted (moved up) but not their assigned values - accessing before assignment gives undefined.
- var is function-scoped, not block-scoped; a var inside a function does not touch a same-named var outside it.
- switch matches a value against cases and needs break to avoid falling through to the next case.
- while checks its condition before running - it can execute zero times.
- do...while checks its condition after running - it always executes at least once.